

Git Cheat Sheet

Comandos del día a día, de lo básico a lo que salva el día. <> = reemplazar.

Configuración inicial

```
git config --global user.name "Jonas"
git config --global user.email "jonas@empresa.com"
git config --global init.defaultBranch main
git config --global pull.rebase true           # pull con rebase en vez de merge
git config --global core.autocrlf input        # Linux/Mac (en Windows: true)
git config --list --show-origin               # ver de dónde sale cada config
git config --global alias.st status           # alias: ahora "git st"
```

Empezar

```
git init                                     # repo nuevo en el directorio actual
git clone <url>                             # clonar
git clone <url> <carpeta> --depth 1          # clon superficial (solo último commit)
git remote -v                               # ver remotos
git remote add origin <url>
git remote set-url origin <url-nueva>
```

Ciclo básico

```
git status                                  # estado (usá -s para versión corta)
git add <archivo>                          # stagear un archivo
git add .                                  # todo lo del directorio actual
git add -p                                 # ¡interactivo! stagear por trozos (hunks)
git commit -m "feat: agrega validación de CBU"
git commit -am "..."                     # add + commit de archivos ya trackeados (no de nuevos)
git commit --amend                         # corregir el último commit (mensaje o contenido)
git commit --amend --no-edit               # sumar cambios al último commit sin tocar el mensaje
```

Zona	Qué es	Cómo salir
Working directory	Tus archivos	<code>git restore <f></code>
Staging area (index)	Lo que va al próximo commit	<code>git restore --staged <f></code>
Repository	Commits	<code>git reset</code> / <code>git revert</code>

Ramas

```
git branch                                # listar locales (-a incluye remotas, -vv con tracking)
git branch <rama>                         # crear
git switch <rama>                         # cambiar (moderno)
git switch -c <rama>                      # crear y cambiar
git checkout -b <rama>                    # equivalente clásico
git switch -c <rama> origin/<rama>        # crear siguiendo una rama remota
git branch -d <rama>                      # borrar (seguro: exige estar mergeada)
git branch -D <rama>                     # borrar a la fuerza
git push origin --delete <rama>          # borrar en el remoto
git branch -m <nuevo-nombre>             # renombrar la rama actual
```

Integrar cambios: merge vs rebase

```
git merge <rama>                         # fusiona y crea commit de merge
git merge --no-ff <rama>                 # fuerza commit de merge (preserva la historia de la rama)
git merge --squash <rama>               # trae todo como un único commit sin stagear

git rebase main                          # reescribe tus commits encima de main (historia lineal)
git rebase -i HEAD~5                     # interactivo: squash, reword, drop, reorder
git rebase --continue | --abort | --skip
```

Regla de oro: nunca hagas `rebase` de commits ya pusheados y compartidos. Rebase para limpiar tu rama local antes del PR; merge para integrar a `main`.

Resolver conflictos

```
git status                # lista los archivos en conflicto
# editar y borrar los marcadores <<<<<< ===== >>>>>>
git add <archivo>
git merge --continue      # o git rebase --continue
git merge --abort         # salir y volver atrás
git checkout --ours <f>   # quedarme con mi versión
git checkout --theirs <f> # quedarme con la del otro lado
git mergetool             # herramienta visual configurada
```

Sincronizar con el remoto

```
git fetch origin          # baja refs sin tocar tu working tree
git fetch --prune         # además limpia ramas remotas borradas
git pull                 # fetch + merge (o rebase si lo configuraste)
git pull --rebase
git push
git push -u origin <rama> # primer push: setea el upstream
git push --force-with-lease # push forzado SEGURO tras un rebase
git push --tags
```

--force-with-lease aborta si alguien pusheó algo que vos no tenés. Nunca uses --force a secas en ramas compartidas.

Deshacer cosas

```
git restore <archivo>      # descartar cambios locales del archivo
git restore --staged <archivo> # sacar del staging (queda modificado)
git restore --source=HEAD~2 <archivo> # traer el archivo como estaba hace 2 commits

git reset --soft HEAD~1    # deshace el commit, deja los cambios STAGEADOS
git reset --mixed HEAD~1   # (default) deshace el commit, cambios sin stagear
git reset --hard HEAD~1    # OJO: borra commit y cambios. Irrecuperable salvo reflog
git reset --hard origin/main # dejar la rama local idéntica al remoto

git revert <sha>           # crea un commit que deshace otro (seguro en ramas compartidas)
git revert -m 1 <sha-merge> # revertir un merge
git clean -fd             # borrar archivos y carpetas no trackeados (-n para simular)
```

El reflog: la red de seguridad

```
git reflog                # historial de TODOS los movimientos de HEAD
git reset --hard HEAD@{3} # volver al estado de hace 3 movimientos
git checkout -b rescate <sha> # recuperar commits "perdidos" de un reset o rebase
```

Stash: guardar trabajo a medias

```
git stash                # guarda cambios trackeados y limpia el working dir
git stash -u             # incluye archivos nuevos (untracked)
git stash push -m "wip login" <archivo>
git stash list
git stash pop            # aplica el último y lo borra de la pila
git stash apply stash@{2} # aplica sin borrarlo
git stash drop stash@{0}
git stash show -p stash@{0} # ver el diff
```

Ver la historia

```
git log --oneline --graph --decorate --all # el más útil, hacete un alias
git log -5 --stat                          # últimos 5 con archivos tocados
git log -p <archivo>                      # historia con diffs de un archivo
git log --follow <archivo>                 # sigue el archivo aunque lo hayan renombrado
git log --author="jonas" --since="2 weeks ago"
git log --grep="JIRA-123"
git log main..feature                     # commits que están en feature y no en main
git show <sha>                             # detalle de un commit
git diff                                  # working vs staging
git diff --staged                         # staging vs último commit
git diff main..feature                    # entre ramas
git diff --name-only HEAD~3               # solo nombres de archivos
git blame <archivo>                       # quién tocó cada línea (-L 10,20 para un rango)
```

Tags

```
git tag v1.2.0 # tag liviano
git tag -a v1.2.0 -m "Release 1.2.0" # anotado (recomendado para releases)
git tag -l "v1.*"
git push origin v1.2.0
git push origin --tags
git tag -d v1.2.0 && git push origin --delete v1.2.0
git describe --tags # versión legible desde el último tag
```

Herramientas de precisión

```
git cherry-pick <sha> # traer un commit puntual a la rama actual
git cherry-pick <sha1>^..<sha2> # un rango
git cherry-pick -n <sha> # sin commitear (deja los cambios en staging)

git bisect start
git bisect bad # el commit actual falla
git bisect good <sha> # este andaba
# Git te va posicionando: probá y marcá good/bad hasta encontrar el culpable
git bisect reset

git worktree add ../hotfix main # segunda copia de trabajo en otra carpeta, mismo repo
git worktree list
git worktree remove ../hotfix
```

.gitignore

```
# Patrones típicos de un proyecto Java
target/
build/
*.class
*.jar
.idea/
*.iml
.env
application-local.yml
!config/.gitkeep # ! niega el patrón anterior

git rm -r --cached . # después de editar .gitignore: destrackea lo ya commiteado
git check-ignore -v <f> # ver qué regla está ignorando un archivo
```

Flujo típico de feature branch

```
git switch main && git pull # 1. partir de main actualizado
git switch -c feature/JIRA-123-validacion # 2. rama nueva
# ... trabajar, commits chicos y atómicos ...
git fetch origin && git rebase origin/main # 3. poner al día antes del PR
git push -u origin feature/JIRA-123-validacion
# 4. abrir el Pull Request, revisión, ajustes
git rebase -i origin/main # 5. limpiar commits (squash de "fix typo")
git push --force-with-lease
# 6. merge del PR, y después:
git switch main && git pull && git branch -d feature/JIRA-123-validacion
```

Convención de mensajes (Conventional Commits)

feat(clientes): agrega endpoint de alta masiva
 fix(cbu): corrige validación de dígito verificador
 refactor(dao): extrae mapper de ResultSet
 docs, test, chore, perf, build, ci

BREAKING CHANGE: en el cuerpo, para cambios incompatibles

Alias que valen la pena

```
git config --global alias.lg "log --oneline --graph --decorate --all"
git config --global alias.st "status -sb"
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.unstage "restore --staged"
git config --global alias.amend "commit --amend --no-edit"
```

Emergencias frecuentes

Problema	Solución
Commiteé en <code>main</code> en vez de una rama	<code>git branch feature && git reset --hard origin/main && git switch feature</code>
Mensaje de commit mal escrito (sin pushear)	<code>git commit --amend</code>
Pusheé algo que no debía	<code>git revert <sha></code> (seguro) o <code>reset + --force-with-lease</code> (solo si es tuya)
Hice <code>reset --hard</code> y perdí trabajo	<code>git reflog</code> y volver al <code>HEAD@{n}</code>
Commiteé una credencial	Rotó la credencial primero ; después <code>git filter-repo</code> o BFG y push forzado
Necesito un cambio de otra rama, solo ese	<code>git cherry-pick <sha></code>
Mi rama quedó atrás de main	<code>git fetch && git rebase origin/main</code>
Quiero ver qué rompió un test	<code>git bisect</code>